

Topic 11
Dynamic Programming – Graph Algorithms
Bellman-Ford, Floyd-Warshall, Warshall's Algorithm

Guide : Dr. Sriramya
Name : Jaasiel J S

1. Bellman-Ford Algorithm

Given a weighted directed graph and a source vertex, find the shortest distance from the source to all other vertices using the Bellman-Ford algorithm. If the graph contains a negative weight cycle, report it.

Example 1:

Input:

Vertices = 5

Edges = [(0,1,-1),(0,2,4),(1,2,3),(1,3,2),(1,4,2),(3,2,5),(3,1,1),(4,3,-3)]

Source = 0

Output:

Vertex	Distance
0	0
1	-1
2	2
3	-2
4	1

Example 2:

Input:

Vertices = 3

Edges = [(0,1,1),(1,2,-1),(2,0,-1)]

Source = 0

Output:

Negative Weight Cycle Detected

main.py	Output
<pre>1 def bellman_ford(vertices, edges, source): 2 INF = float("inf") 3 dist = [INF] * vertices 4 dist[source] = 0 5 6 for _ in range(vertices - 1): 7 updated = False 8 for u, v, w in edges: 9 if dist[u] != INF and dist[u] + w < dist[v]: 10 dist[v] = dist[u] + w 11 updated = True 12 if not updated: 13 break 14 15 for u, v, w in edges: 16 if dist[u] != INF and dist[u] + w < dist[v]: 17 return None 18 return dist 19 20 edges1 = [(0,1,-1),(0,2,4),(1,2,3),(1,3,2), 21 (1,4,2),(3,2,5),(3,1,1),(4,3,-3)] 22 result1 = bellman_ford(5, edges1, 0) 23 24 print("Example 1") 25 print("Vertex\tDistance") 26 for v, d in enumerate(result1): 27 print(f"{v}\t{d}") 28 29 edges2 = [(0,1,1),(1,2,-1),(2,0,-1)] 30 result2 = bellman_ford(3, edges2, 0) 31 32 print("\nExample 2") 33 if result2 is None: 34 print("Negative Weight Cycle Detected") 35 else: 36 print(result2)</pre>	<pre>Example 1 Vertex Distance 0 0 1 -1 2 2 3 -2 4 1 Example 2 Negative Weight Cycle Detected === Code Execution Successful ===</pre>

2. Floyd-Warshall Algorithm

Given a weighted graph represented by an adjacency matrix, find the shortest distances between every pair of vertices using the Floyd-Warshall algorithm.

Example 1:

Input:

```
[
[0,5,INF,10],
[INF,0,3,INF],
[INF,INF,0,1],
[INF,INF,INF,0]
]
```

Output:

```
[
[0,5,8,9],
```

```
[INF,0,3,4],  
[INF,INF,0,1],  
[INF,INF,INF,0]  
]
```

Example 2:

Input:

```
[  
[0,2,INF],  
[INF,0,4],  
[1,INF,0]  
]
```

Output:

```
[  
[0,2,6],  
[5,0,4],  
[1,3,0]  
]
```

main.py	Output
<pre> 1 INF = float("inf") 2 3 def floyd_warshall(graph): 4 n = len(graph) 5 dist = [row[:] for row in graph] 6 7 for k in range(n): 8 for i in range(n): 9 for j in range(n): 10 if dist[i][k] != INF and dist[k][j] != INF: 11 dist[i][j] = min(12 dist[i][j], 13 dist[i][k] + dist[k][j] 14) 15 return dist 16 17 def show(matrix): 18 for row in matrix: 19 values = ["INF" if x == INF else str(x) for x in row] 20 print("[" + ", ".join(values) + "]") 21 22 graph1 = [23 [0, 5, INF, 10], 24 [INF, 0, 3, INF], 25 [INF, INF, 0, 1], 26 [INF, INF, INF, 0] 27] 28 29 graph2 = [30 [0, 2, INF], 31 [INF, 0, 4], 32 [1, INF, 0] 33] 34 35 print("Example 1") 36 show(floyd_warshall(graph1)) 37 print("\nExample 2") 38 show(floyd_warshall(graph2)) </pre>	<pre> Example 1 [0,5,8,9] [INF,0,3,4] [INF,INF,0,1] [INF,INF,INF,0] Example 2 [0,2,6] [5,0,4] [1,3,0] === Code Execution Successful === </pre>

3. Warshall's Algorithm

Given a directed graph represented by an adjacency matrix, determine whether a path exists between every pair of vertices. Compute the transitive closure of the graph using Warshall's algorithm.

Example 1:

Input:

```

[
[0,1,0,0],
[0,0,1,0],
[0,0,0,1],
[0,0,0,0]
]

```

Output:

```

[
[0,1,1,1],

```

```
[0,0,1,1],  
[0,0,0,1],  
[0,0,0,0]  
]
```

Example 2:

Input:

```
[  
[0,1,1],  
[0,0,1],  
[1,0,0]  
]
```

Output:

```
[  
[1,1,1],  
[1,1,1],  
[1,1,1]  
]
```

main.py	Output
<pre>1 def warshall(graph): 2 n = len(graph) 3 reach = [row[:] for row in graph] 4 5 for k in range(n): 6 for i in range(n): 7 for j in range(n): 8 reach[i][j] = int(9 reach[i][j] or 10 (reach[i][k] and reach[k][j]) 11) 12 return reach 13 14 def show(matrix): 15 for row in matrix: 16 print "[" + ", ".join(map(str, row)) + "]" 17 18 graph1 = [19 [0,1,0,0], 20 [0,0,1,0], 21 [0,0,0,1], 22 [0,0,0,0] 23] 24 25 graph2 = [26 [0,1,1], 27 [0,0,1], 28 [1,0,0] 29] 30 31 print("Example 1") 32 show(warshall(graph1)) 33 print("\nExample 2") 34 show(warshall(graph2))</pre>	<pre>Example 1 [0,1,1,1] [0,0,1,1] [0,0,0,1] [0,0,0,0] Example 2 [1,1,1] [1,1,1] [1,1,1] === Code Execution Successful ===</pre>

4. Find the City with the Smallest Number of Reachable Cities

Given a weighted graph and a distance threshold, find the city that can reach the smallest number of other cities within the threshold distance. Use Floyd-Warshall algorithm to compute all-pairs shortest paths.

Example 1:

Input:

```
n=4  
edges=[[0,1,3],[1,2,1],[2,3,4],[0,3,7]]  
threshold=4
```

Output:

3

Explanation:

City 3 can reach the fewest cities within the given threshold distance.

Example 2:

Input:

```
n=5  
edges=[[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]  
threshold=2
```

Compute reachable cities within distance ≤ 2 :

Output:

0

main.py	Output
<pre> 1 INF = float("inf") 2 3 def find_city(n, edges, threshold): 4 dist = [[INF] * n for _ in range(n)] 5 for i in range(n): 6 dist[i][i] = 0 7 8 for u, v, w in edges: 9 dist[u][v] = min(dist[u][v], w) 10 dist[v][u] = min(dist[v][u], w) 11 12 for k in range(n): 13 for i in range(n): 14 for j in range(n): 15 dist[i][j] = min(16 dist[i][j], 17 dist[i][k] + dist[k][j] 18) 19 20 counts = [] 21 for i in range(n): 22 count = sum(23 1 for j in range(n) 24 if i != j and dist[i][j] <= threshold 25) 26 counts.append(count) 27 28 minimum = min(counts) 29 return counts.index(minimum) 30 31 print("Example 1") 32 print(find_city(33 4, 34 [[0,1,3],[1,2,1],[2,3,4],[0,3,7]], 35 4 36)) 37 38 print("\nExample 2") 39 print(find_city(40 5, 41 [[0,1,2],[0,4,8],[1,2,3], 42 [1,4,2],[2,3,1],[3,4,1]], 43 2 44)) </pre>	<pre> Example 1 3 Example 2 0 === Code Execution Successful === </pre>

5. Network Delay Time

A signal is sent from a source node to all nodes in a network. Find the minimum time required for all nodes to receive the signal using the Bellman-Ford algorithm. Return -1 if some nodes cannot be reached.

Example 1:

Input:

```
times=[[2,1,1],[2,3,1],[3,4,1]]
```

```
n=4
```

```
k=2
```

Output:

2

Explanation:

The signal reaches all nodes in 2 units of time.

Example 2:

Input:

```
times=[[1,2,1]]
```

```
n=2
```

```
k=2
```

Output:

-1

main.py	Output
<pre>1 def network_delay_time(times, n, k): 2 INF = float("inf") 3 dist = [INF] * (n + 1) 4 dist[k] = 0 5 6 for _ in range(n - 1): 7 updated = False 8 for u, v, w in times: 9 if dist[u] != INF and dist[u] + w < dist[v]: 10 dist[v] = dist[u] + w 11 updated = True 12 if not updated: 13 break 14 15 answer = max(dist[1:]) 16 return -1 if answer == INF else answer 17 18 print("Example 1") 19 print(network_delay_time(20 [[2,1,1],[2,3,1],[3,4,1]], 4, 2 21)) 22 23 print("\nExample 2") 24 print(network_delay_time(25 [[1,2,1]], 2, 2 26))</pre>	<pre>Example 1 2 Example 2 -1 === Code Execution Successful ===</pre>

6. Cheapest Flight Within K Stops

Given a list of flights represented as (source, destination, cost), find the minimum cost to travel from the source city to the destination city with at most K stops using the Bellman-Ford algorithm.

Example 1:

Input:

```
n = 4
flights = [[0,1,100],[1,2,100],[2,3,100],[0,3,500]]
src = 0
dst = 3
K = 1
```

Output:

500

Explanation:

The route $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ requires 2 stops and is not allowed.

Example 2:

Input:

```
n = 4
flights = [[0,1,100],[1,2,100],[2,3,100],[0,3,500]]
src = 0
dst = 3
K = 2
```

Output:

300

Explanation:

The route $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ costs 300 and uses exactly 2 stops.

main.py	Output
<pre> 1 def cheapest_flight(n, flights, src, dst, k): 2 INF = float("inf") 3 dist = [INF] * n 4 dist[src] = 0 5 6 # At most K stops means at most K+1 edges. 7 for _ in range(k + 1): 8 next_dist = dist[:] 9 for u, v, price in flights: 10 if dist[u] != INF: 11 next_dist[v] = min(12 next_dist[v], 13 dist[u] + price 14) 15 dist = next_dist 16 17 return -1 if dist[dst] == INF else dist[dst] 18 19 flights = [20 [0,1,100],[1,2,100], 21 [2,3,100],[0,3,500] 22] 23 24 print("Example 1") 25 print(cheapest_flight(4, flights, 0, 3, 1)) 26 27 print("\nExample 2") 28 print(cheapest_flight(4, flights, 0, 3, 2)) </pre>	<pre> Example 1 500 Example 2 300 === Code Execution Successful === </pre>

7. Detect Negative Weight Cycle

Given a weighted directed graph, determine whether it contains a negative weight cycle using the Bellman-Ford algorithm.

Example 1:

Input:

Vertices = 4

Edges = [(0,1,1),(1,2,-1),(2,3,-1),(3,0,-1)]

Output:

Negative Weight Cycle Exists

Explanation:

The cycle weight is $1 + (-1) + (-1) + (-1) = -2$.

Example 2:

Input:

Vertices = 4

Edges = [(0,1,4),(0,2,5),(1,2,-2),(2,3,3)]

Output:

No Negative Weight Cycle

main.py	Output
<pre>1 def has_negative_cycle(vertices, edges): 2 # Start every vertex at 0 so cycles in any 3 # disconnected component are also detected. 4 dist = [0] * vertices 5 6 for i in range(vertices): 7 updated = False 8 for u, v, w in edges: 9 if dist[u] + w < dist[v]: 10 dist[v] = dist[u] + w 11 updated = True 12 if i == vertices - 1: 13 return True 14 if not updated: 15 return False 16 return False 17 18 edges1 = [19 (0,1,1),(1,2,-1), 20 (2,3,-1),(3,0,-1) 21] 22 edges2 = [23 (0,1,4),(0,2,5), 24 (1,2,-2),(2,3,3) 25] 26 27 print("Example 1") 28 print("Negative Weight Cycle Exists" 29 if has_negative_cycle(4, edges1) 30 else "No Negative Weight Cycle") 31 32 print("\nExample 2") 33 print("Negative Weight Cycle Exists" 34 if has_negative_cycle(4, edges2) 35 else "No Negative Weight Cycle")</pre>	<pre>Example 1 Negative Weight Cycle Exists Example 2 No Negative Weight Cycle === Code Execution Successful ===</pre>

8. Determine Reachability Between Two Vertices

Given a directed graph represented as an adjacency matrix and two vertices u and v , determine whether a path exists from u to v using Warshall's Algorithm.

Example 1:

Input:

Graph =

```
[  
  [0,1,0],  
  [0,0,1],  
  [0,0,0]  
]
```

$u = 0$

$v = 2$

Output:

Path Exists

Explanation:

A path exists from 0 to 2 through vertex 1.

Example 2:

Input:

Graph =

```
[  
  [0,1,0],  
  [0,0,0],  
  [1,0,0]  
]
```

$u = 1$

$v = 2$

Output:

Path Does Not Exist

main.py	Output
<pre> 1 def warshall(graph): 2 n = len(graph) 3 reach = [row[:] for row in graph] 4 5 for k in range(n): 6 for i in range(n): 7 for j in range(n): 8 reach[i][j] = int(9 reach[i][j] or 10 (reach[i][k] and reach[k][j])) 11 12 return reach 13 14 def path_exists(graph, u, v): 15 return bool(warshall(graph)[u][v]) 16 17 graph1 = [18 [0,1,0], 19 [0,0,1], 20 [0,0,0] 21] 22 23 graph2 = [24 [0,1,0], 25 [0,0,0], 26 [1,0,0] 27] 28 29 print("Example 1") 30 print("Path Exists" if path_exists(graph1, 0, 2) 31 else "Path Does Not Exist") 32 33 print("\nExample 2") 34 print("Path Exists" if path_exists(graph2, 1, 2) 35 else "Path Does Not Exist") </pre>	<pre> Example 1 Path Exists Example 2 Path Does Not Exist === Code Execution Successful === </pre>

9. Count Reachable Vertices from a Source

Given a directed graph represented by an adjacency matrix, find the number of vertices reachable from a specified source vertex using the transitive closure generated by Warshall's Algorithm. Do not count the source vertex itself.

Example 1:

Input:

Graph =

```
[
  [0,1,0,0],
  [0,0,1,0],
  [0,0,0,1],
  [0,0,0,0]
]
```

Source = 0

Output:

3

Explanation:

Vertices 1, 2, and 3 are reachable from vertex 0.

Example 2:

Input:

Graph =

```
[  
  [0,1,1],  
  [0,0,1],  
  [1,0,0]  
]
```

Source = 1

Output:

2

Explanation:

Vertices 0 and 2 are reachable from vertex 1.

main.py	Output
<pre>1 def warshall(graph): 2 n = len(graph) 3 reach = [row[:] for row in graph] 4 5 for k in range(n): 6 for i in range(n): 7 for j in range(n): 8 reach[i][j] = int(9 reach[i][j] or 10 (reach[i][k] and reach[k][j]) 11) 12 return reach 13 14 def count_reachable(graph, source): 15 reach = warshall(graph) 16 return sum(17 1 for v in range(len(graph)) 18 if v != source and reach[source][v] 19) 20 21 graph1 = [22 [0,1,0,0], 23 [0,0,1,0], 24 [0,0,0,1], 25 [0,0,0,0] 26] 27 28 graph2 = [29 [0,1,1], 30 [0,0,1], 31 [1,0,0] 32] 33 34 print("Example 1") 35 print(count_reachable(graph1, 0)) 36 37 print("\nExample 2") 38 print(count_reachable(graph2, 1))</pre>	<pre>Example 1 3 Example 2 2 === Code Execution Successful ===</pre>

10. Find the Diameter of a Graph

Given a weighted graph represented by an adjacency matrix, find the diameter of the graph using the Floyd-Warshall algorithm. The diameter is the maximum shortest-path distance between any pair of vertices.

Example 1:

Input:

Graph =
[
[0,2,INF],
[2,0,3],
[INF,3,0]
]

Output:

5

Explanation:

The shortest path from vertex 0 to vertex 2 is $0 \rightarrow 1 \rightarrow 2$ with cost 5.

Example 2:

Input:

Graph =
[
[0,1,4],
[1,0,2],
[4,2,0]
]

Output:

3

Explanation:

The shortest path from vertex 0 to vertex 2 is $0 \rightarrow 1 \rightarrow 2$ with cost 3, which is the graph diameter.

main.py	Output
<pre> 1 INF = float("inf") 2 3 def graph_diameter(graph): 4 n = len(graph) 5 dist = [row[:] for row in graph] 6 7 for k in range(n): 8 for i in range(n): 9 for j in range(n): 10 if dist[i][k] != INF and dist[k][j] != INF: 11 dist[i][j] = min(12 dist[i][j], 13 dist[i][k] + dist[k][j] 14) 15 16 if any(dist[i][j] == INF 17 for i in range(n) for j in range(n)): 18 return INF 19 20 return max(21 dist[i][j] 22 for i in range(n) 23 for j in range(n) 24) 25 26 graph1 = [27 [0,2,INF], 28 [2,0,3], 29 [INF,3,0] 30] 31 32 graph2 = [33 [0,1,4], 34 [1,0,2], 35 [4,2,0] 36] 37 38 print("Example 1") 39 print(graph_diameter(graph1)) 40 print("\nExample 2") 41 print(graph_diameter(graph2)) </pre>	<pre> Example 1 5 Example 2 3 === Code Execution Successful === </pre>

11. Find the Shortest Distance Between Two Vertices

Given a weighted graph represented by an adjacency matrix and two vertices u and v , find the shortest distance between them using the Floyd-Warshall algorithm.

Example 1:

Input:

Graph =

```

[
  [0,3,INF,7],
  [8,0,2,INF],
  [5,INF,0,1],
  [2,INF,INF,0]
]
```

$u = 0$

$v = 3$

Output:

6

Explanation:

Shortest path: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

Distance = $3 + 2 + 1 = 6$

Example 2:

Input:

Graph =

```
[
  [0,4,INF],
  [INF,0,6],
  [2,INF,0]
]
```

u = 2

v = 1

Output:

6

Explanation:

Shortest path: $2 \rightarrow 0 \rightarrow 1$

Distance = $2 + 4 = 6$

main.py	Output
<pre>1 INF = float("inf") 2 3 def shortest_distance(graph, u, v): 4 n = len(graph) 5 dist = [row[:] for row in graph] 6 7 for k in range(n): 8 for i in range(n): 9 for j in range(n): 10 if dist[i][k] != INF and dist[k][j] != INF: 11 dist[i][j] = min(12 dist[i][j], 13 dist[i][k] + dist[k][j] 14) 15 return dist[u][v] 16 17 graph1 = [18 [0,3,INF,7], 19 [8,0,2,INF], 20 [5,INF,0,1], 21 [2,INF,INF,0] 22] 23 24 graph2 = [25 [0,4,INF], 26 [INF,0,6], 27 [2,INF,0] 28] 29 30 print("Example 1") 31 print(shortest_distance(graph1, 0, 3)) 32 print("\nExample 2") 33 print(shortest_distance(graph2, 2, 1))</pre>	<pre>Example 1 6 Example 2 6 === Code Execution Successful ===</pre>

12. Find a Vertex Reachable from Every Other Vertex

Given a directed graph represented by an adjacency matrix, find a vertex that is reachable from every other vertex using Warshall's Algorithm. If multiple vertices satisfy the condition, return the smallest indexed vertex.

Example 1:

Input:

Graph =
[
[0,1,1],
[0,0,1],
[0,0,0]
]

Output:

2

Explanation:

Vertices 0 and 1 can reach vertex 2.

Example 2:

Input:

Graph =
[
[0,1,0],
[0,0,1],
[1,0,0]
]

Output:

0

Explanation:

All vertices are reachable from every other vertex.
Return the smallest indexed vertex.

main.py	Output
<pre> 1 def warshall(graph): 2 n = len(graph) 3 reach = [row[:] for row in graph] 4 5 for k in range(n): 6 for i in range(n): 7 for j in range(n): 8 reach[i][j] = int(9 reach[i][j] or 10 (reach[i][k] and reach[k][j])) 11 12 return reach 13 14 def reachable_from_every_vertex(graph): 15 reach = warshall(graph) 16 n = len(graph) 17 18 for v in range(n): 19 if all(i == v or reach[i][v] for i in range(n)): 20 return v 21 return -1 22 23 graph1 = [24 [0,1,1], 25 [0,0,1], 26 [0,0,0] 27] 28 29 graph2 = [30 [0,1,0], 31 [0,0,1], 32 [1,0,0] 33] 34 35 print("Example 1") 36 print(reachable_from_every_vertex(graph1)) 37 print("\nExample 2") 38 print(reachable_from_every_vertex(graph2)) </pre>	<pre> Example 1 2 Example 2 0 === Code Execution Successful === </pre>

13. Find the Vertex with Maximum Reachability

Given a directed graph represented by an adjacency matrix, find the vertex that can reach the maximum number of other vertices using Warshall's Algorithm. If there is a tie, return the smallest indexed vertex.

Example 1:

Input:

Graph =

```

[
  [0,1,1,0],
  [0,0,1,0],
  [0,0,0,1],
  [0,0,0,0]
]
```

Output:

0

Explanation:

Vertex 0 can reach vertices 1, 2 and 3.

Example 2:**Input:**

Graph =

```
[  
  [0,1,0],  
  [0,0,1],  
  [1,0,0]  
]
```

Output:

0

Explanation:

All vertices can reach two other vertices.

Return the smallest indexed vertex.

main.py	Output
<pre> 1 def warshall(graph): 2 n = len(graph) 3 reach = [row[:] for row in graph] 4 5 for k in range(n): 6 for i in range(n): 7 for j in range(n): 8 reach[i][j] = int(9 reach[i][j] or 10 (reach[i][k] and reach[k][j])) 11) 12 return reach 13 14 def max_reachability_vertex(graph): 15 reach = warshall(graph) 16 n = len(graph) 17 best_vertex = 0 18 best_count = -1 19 20 for i in range(n): 21 count = sum(22 1 for j in range(n) 23 if i != j and reach[i][j] 24) 25 if count > best_count: 26 best_count = count 27 best_vertex = i 28 29 return best_vertex 30 31 graph1 = [32 [0,1,1,0], 33 [0,0,1,0], 34 [0,0,0,1], 35 [0,0,0,0] 36] 37 38 graph2 = [39 [0,1,0], 40 [0,0,1], 41 [1,0,0] 42] 43 44 print("Example 1") 45 print(max_reachability_vertex(graph1)) 46 print("\nExample 2") 47 print(max_reachability_vertex(graph2)) </pre>	<pre> Example 1 0 Example 2 0 === Code Execution Successful === </pre>

14. Find the Diameter of a Weighted Graph

Given a weighted graph represented by an adjacency matrix, find the diameter of the graph using the Floyd-Warshall algorithm. The diameter is the maximum shortest-path distance between any pair of vertices.

Example 1:

Input:

Graph =

```

[
  [0,2,INF],
  [2,0,4],
  [INF,4,0]
]
```

Output:

6

Explanation:

Shortest path from 0 to 2 is:

$0 \rightarrow 1 \rightarrow 2 = 6$

Diameter = 6

Example 2:

Input:

Graph =

[
[0,1,5],
[1,0,2],
[5,2,0]
]

Output:

3

Explanation:

Shortest path from 0 to 2 is:

$0 \rightarrow 1 \rightarrow 2 = 3$

Diameter = 3

main.py	Output
<pre> 1 INF = float("inf") 2 3 def weighted_graph_diameter(graph): 4 n = len(graph) 5 dist = [row[:] for row in graph] 6 7 for k in range(n): 8 for i in range(n): 9 for j in range(n): 10 if dist[i][k] != INF and dist[k][j] != INF: 11 dist[i][j] = min(12 dist[i][j], 13 dist[i][k] + dist[k][j] 14) 15 16 if any(dist[i][j] == INF 17 for i in range(n) for j in range(n)): 18 return INF 19 20 return max(21 dist[i][j] 22 for i in range(n) 23 for j in range(n) 24) 25 26 graph1 = [27 [0,2,INF], 28 [2,0,4], 29 [INF,4,0] 30] 31 32 graph2 = [33 [0,1,5], 34 [1,0,2], 35 [5,2,0] 36] 37 38 print("Example 1") 39 print(weighted_graph_diameter(graph1)) 40 print("\nExample 2") 41 print(weighted_graph_diameter(graph2)) </pre>	<pre> Example 1 6 Example 2 3 === Code Execution Successful === </pre>

15. Check Whether a Graph is Strongly Connected

Given a directed graph represented by an adjacency matrix, determine whether every vertex is reachable from every other vertex using Warshall's Algorithm.

Example 1:

Input:

Graph =

```

[
  [0,1,0],
  [0,0,1],
  [1,0,0]
]
```

Output:

Strongly Connected

Explanation:

Every vertex can reach every other vertex.

Example 2:

Input:

Graph =

```
[  
  [0,1,0],  
  [0,0,1],  
  [0,0,0]  
]
```

Output:

Not Strongly Connected

Explanation:

Vertex 2 cannot reach vertices 0 and 1.

main.py	Output
<pre>1 def warshall(graph): 2 n = len(graph) 3 reach = [row[:] for row in graph] 4 5 for k in range(n): 6 for i in range(n): 7 for j in range(n): 8 reach[i][j] = int(9 reach[i][j] or 10 (reach[i][k] and reach[k][j])) 11) 12 return reach 13 14 def is_strongly_connected(graph): 15 reach = warshall(graph) 16 n = len(graph) 17 18 return all(19 i == j or reach[i][j] 20 for i in range(n) 21 for j in range(n) 22) 23 24 graph1 = [25 [0,1,0], 26 [0,0,1], 27 [1,0,0] 28] 29 30 graph2 = [31 [0,1,0], 32 [0,0,1], 33 [0,0,0] 34] 35 36 print("Example 1") 37 print("Strongly Connected" 38 if is_strongly_connected(graph1) 39 else "Not Strongly Connected") 40 41 print("\nExample 2") 42 print("Strongly Connected" 43 if is_strongly_connected(graph2) 44 else "Not Strongly Connected")</pre>	<pre>Example 1 Strongly Connected Example 2 Not Strongly Connected === Code Execution Successful ===</pre>

